

Recherche opérationnelle : Projet 1

Luigi VUACHET – `luigi.vuachet.etu@univ-lille.fr`
Kobla LEGBEDJE – `kobla.legbedje.etu@univ-lille.fr`

13 janvier 2025

Introduction

La recherche opérationnelle a pour objectif de concevoir des approches efficaces destinées à résoudre divers problèmes d'optimisation, qu'il s'agisse d'augmenter ou de réduire des résultats spécifiques. Parmi les nombreuses problématiques traitées, on peut citer la recherche du chemin le plus court entre deux lieux, la gestion des flux de transport, ainsi que l'optimisation de la répartition des ressources. Dans ce projet, notre attention se portera sur l'optimisation d'un modèle de distribution.

Pour ce faire, nous ferons appel à AMPL, un logiciel conçu pour la modélisation et la résolution de problèmes d'optimisation. Dans un premier temps, nous exposerons le problème de minimisation que nous devons traiter ainsi que sa modélisation. Par la suite, nous partagerons les résultats obtenus.

Présentation du problème

Une entreprise dispose de deux usines et de quatre dépôts. Elle vend des produits à six clients qui sont fournis à partir des usines et des dépôts.

Précisons que :

- les clients sont notés de C1 à C6.
- il y a une usine à Liverpool et une à Brighton.
- il y a quatre dépôts : Newcastle, Birmingham, Londres et Exeter

Présentons les données :

Les coûts de distributions sont donnés dans le tableau ci-dessous (en livres par tonnes, un zéro signifie que le fournisseur F ne livre pas à C).

fournit à	Newcastle	Birmingham	Londres	Exeter	C1	C2	C3	C4	C5	C6
Liverpool	0.5	0.5	1	0.2	1	0	1.5	2	0	1.0
Brighton	0	0.3	0.5	0.2	2	0	0	0	0	0
Newcastle	0	0	0	0	0	1.5	0.5	1.5	0	1
Birmingham	0	0	0	0	1	0.5	0.5	1	0.5	0
Londres	0	0	0	0	0	1.5	2	0	0.5	1.5
Exeter	0	0	0	0	0	0	0.2	1.5	0.5	1.5

Chaque usine a une capacité de production limitée :

	débit mensuel (en tonnes)
Newcastle	70000
Birmingham	50000
Londres	100000
Exeter	40000

Les clients ont des besoins mensuels à satisfaire absolument :

	besoin mensuel (en tonnes)
C1	50000
C2	10000
C3	40000
C4	35000
C5	60000
C6	20000

Les clients ont pour certains des préférences de livraison selon les fournisseurs :

Client	Fournisseur(s) préféré(s)
C1	usine de Liverpool
C2	dépôt de Newcastle
C3	pas de préférence
C4	pas de préférence
C5	dépôt de Birmingham
C6	dépôts d'Exeter et de Londres

Objectif

Dans un premier temps, nous cherchons à déterminer le schéma de distribution qui minimise le coût total. Ensuite, nous analyserons l'effet d'un accroissement des capacités des usines et des dépôts sur les coûts et le schéma de distribution. Enfin, nous prendrons en compte les préférences des clients et le coût qui en découle.

Présentation du code AMPL

Fichier : exercice_2.mod, exercice_2.dat, exercice_2.run, exercice_2.log

Ensembles

Fichier .mod

```
set Clients; # Liste des clients
set Depots; # Liste des dépôts
set Usines; # Liste des usines
set Usines_Depots := Usines union Depots;
set Depots_Clients := Depots union Clients;

# Ici, on crée deux ensembles, Usines_Depots et Depots_Clients,
# pour simplifier l'écriture des contraintes par la suite.

set CAT := Usines union Depots union Clients;

# CAT sert simplement à contenir les usines, dépôts et clients pour
# construire les ensembles ci-dessous.

set RouteUC within CAT cross CAT;
# RouteUC correspond à tous les chemins de livraison possibles
# entre les usines et les clients.
set ZUC := (Usines cross Clients) diff RouteUC;
set RouteDC within CAT cross CAT;
# RouteDC correspond à tous les chemins de livraison possibles
# entre les dépôts et les clients.
set ZDC := (Depots cross Clients) diff RouteDC;
set RouteUD within CAT cross CAT;
# RouteUD correspond à tous les chemins de livraison possibles
# entre les usines et les dépôts.
set ZUD := (Usines cross Depots) diff RouteUD;

# ZUC, ZDC et ZUD correspondent aux chemins de livraison
# impossibles respectivement contraires aux ensembles RouteUC,
# RouteDC et RouteUD.
```

Fichier .dat

```
set Clients := C1 C2 C3 C4 C5 C6;
set Depots := Newcastle Birmingham Londres Exeter;
set Usines := Liverpool Brighton;
set RouteUC := (Liverpool,C1) (Liverpool,C3) (Liverpool,C4) (Liverpool,C6)
               (Brighton,C1);
set RouteDC := (Newcastle,C2) (Newcastle,C3) (Newcastle,C4) (Newcastle,C6)
               (Birmingham,C1) (Birmingham,C2) (Birmingham,C3) (Birmingham,C4)
               (Birmingham,C5) (Londres,C2) (Londres,C3) (Londres,C5) (Londres,C6)
               (Exeter,C3) (Exeter,C4) (Exeter,C5) (Exeter,C6);

set RouteUD := (Liverpool,Newcastle) (Liverpool,Birmingham)
               (Liverpool,Londres) (Liverpool,Exeter) (Brighton,Birmingham)
               (Brighton,Londres) (Brighton,Exeter);
```

Paramètres

Fichier .mod

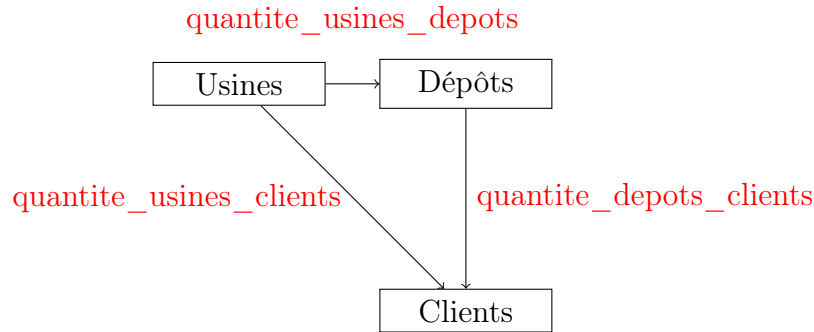
```
param capa_depart{Usines_Depots};
# Contient les capacités de production des usines et les débits
# mensuels limités des dépôts.
param besoin_clients{Clients};
# Contient les besoins des clients.
param prix_routes{Usines_Depots,Depots_Clients};
# Stocke tous les prix.
```

Fichier .dat

```
param capa_depart := Liverpool 150000 Brighton 200000 Newcastle 70000 Birmingham 50000 Londres 100000 Exeter 40000;
param besoin_clients := C1 50000 C2 10000 C3 40000 C4 35000 C5 60000 C6 20000;
param prix_routes : Newcastle Birmingham Londres Exeter C1 C2 C3 C4 C5 C6 :=
Liverpool      0.5      0.5      1      0.2  1      0  1.5  2      0  1.0
Brighton       0        0.3      0.5  0.2  2      0  0      0  0      0
Newcastle      0        0        0      0      0  1.5  0.5  1.5  0      1
Birmingham    0        0        0      0      0  1      0.5  0.5  1      0.5  0
Londres        0        0        0      0      0  1.5  2      0  0.5  1.5
Exeter         0        0        0      0      0  0      0.2  1.5  0.5  1.5;
```

Variables

Le schéma de distribution est décomposé en trois variables (en rouge) à plusieurs dimensions (dépendant du nombre d'usines, de dépôts et de clients) :



Contraintes

1. Pour chaque dépôt, la quantité de produit envoyée des usines au dépôt doit être égale à la quantité de produit envoyée du dépôt aux clients : cela permet d'éviter qu'il n'y ait des produits qui restent dans un dépôt non livré.

```

subject to bilan{d in Depots}:
  sum {u in Usines} quantite_usines_depots[u,d]
  = sum {c in Clients} quantite_depots_clients[d,c] ;
  
```

2. Chaque usine doit produire en dessous de sa capacité de production.

```

subject to capacite_limite{u in Usines}:
  sum {d in Depots} quantite_usines_depots[u,d]
  + sum {c in Clients} quantite_usines_clients[u,c] <=
  capa_depart[u];
  
```

3. Chaque entrepôt a un débit limité. Comme la quantité qui entre et qui sort est égale, il suffit de vérifier que l'une de ces quantités ne dépasse pas le débit mensuel (Contrainte 1).

```

subject to debit_limite{d in Depots}:
  sum {u in Usines} quantite_usines_depots[u,d] <=
  capa_depart[d];
  
```

4. Les clients doivent satisfaire leurs besoins mensuels.

```

subject to satisfaction_besoin{c in Clients}:
  sum {d in Depots} quantite_depots_clients[d,c]
  + sum {u in Usines} quantite_usines_clients[u,c] ==
  besoin_clients[c];
  
```

5. Les quantités associées aux chemins de livraison qui ne sont pas possibles doivent être nulles.

```

subject to zero_DC {(p,c) in ZDC} :
  
```

```

quantite_depots_clients[p,c] =0;

subject to zero_UC {(d,k) in ZUC}:
    quantite_usines_clients[d,k] = 0;

subject to zero_UD {(k,l) in ZUD}:
    quantite_usines_depots[k,l] = 0;

```

Minimisation

On minimise le coût associé aux chemins de livraison et aux quantités correspondantes.

```

minimize cout:
    sum{u1 in Usines,d1 in Depots}
        quantite_usines_depots[u1,d1]*prix_routes[u1,d1]
+ sum{d2 in Depots,c2 in Clients}
        quantite_depots_clients[d2,c2]*prix_routes[d2,c2]
+ sum{u3 in Usines,c3 in Clients}
        quantite_usines_clients[u3,c3]*prix_routes[u3,c3];

```

Résultats

Le coût est de **198500**.

quantite_usines_depots	Birmingham	Exeter	Londres	Newcastle
Liverpool	0	40000	0	0
Brighton	50000	0	55000	0

TABLE 1: Quantité des usines aux dépôts

quantite_depots_clients	C1	C2	C3	C4	C5	C6
Newcastle	0	0	0	0	0	0
Birmingham	0	10000	0	35000	5000	0
Londres	0	0	0	0	55000	0
Exeter	0	0	40000	0	0	0

TABLE 2: Quantité des dépôts aux clients

quantite_usines_clients	C1	C2	C3	C4	C5	C6
Liverpool	50000	0	0	0	0	20000
Brighton	0	0	0	0	0	0

TABLE 3: Quantité des usines aux clients

Conclusion

Le coût de distribution associé au schéma ci-dessous est de **198 500** livres. Aucun produit n'est fourni au dépôt de Newcastle.

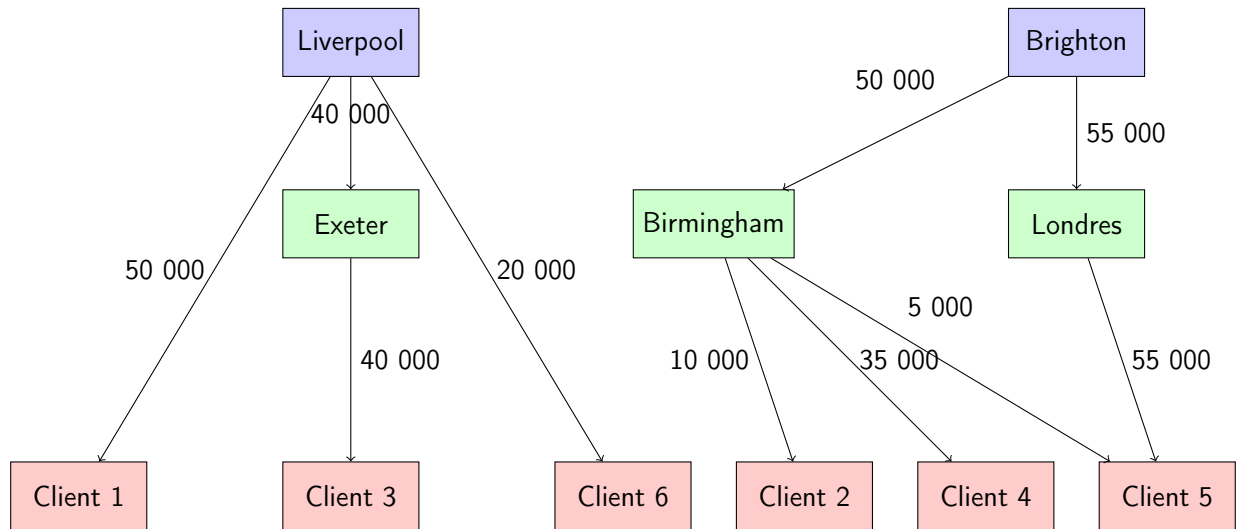


FIGURE 1 – Schéma de distribution des produits (quantités en tonnes)

Accroissement des capacités des usines et dépôts

On constate que lorsque l'on augmente uniquement les capacités de production des usines, le coût de transport ne diminue pas et le schéma de distribution ne change pas. En revanche, si on augmente les capacités mensuels pour Birmingham (de 50 000 à 55 000) et Exeter (de 40 000 à 45 000), les coûts diminuent de 198 500 à 196 000) et les quantités dans le schéma changent. L'augmentation des capacités a donc modifié la répartition des flux, et le dépôt d'Exeter joue désormais un rôle plus important en assurant un débit supplémentaire vers le client 5.

Prise en compte des préférences

Considérons un nouveau paramètre :

Fichier .mod

```
param preferences{Clients, Usines_Depots};
```

Fichier .dat

```
param preferences : Liverpool Brighton Newcastle Birmingham Londres Exeter
:=
      C1          1          0          0          0          0          0
      C2          0          0          1          0          0          0
      C3          1          1          1          1          1          1
      C4          1          1          1          1          1          1
      C5          0          0          0          1          0          0
      C6          0          0          0          0          1          1;
```

Maximisons la satisfaction, qui correspond à la **quantité totale de produits qui passent par les fournisseurs préférés des clients** :

```
maximize satisfaction_preferences:
sum{u in Usines, c in Clients}
    quantite_usines_clients[u, c] * preferences[c, u]
+ sum{d in Depots, c in Clients}
    quantite_depots_clients[d, c] * preferences[c, d];
```

Résultats

Nous remarquons qu'il n'existe pas de schéma de distribution qui respecte à la lettre les préférences des clients tout en satisfaisant leurs besoins. En effet, pour cela, il faudrait que **satisfaction_preferences** soit égale à 215 000 (la quantité totale de produits si les préférences et les besoins des clients sont respectés).

Or, **satisfaction_preferences** = 205000 avec un coût de **285000**

quantite_usines_depots	Birmingham	Exeter	Londres	Newcastle
Liverpool	0	0	15000	70000
Brighton	50000	0	15000	0

TABLE 4: Quantité des usines aux dépôts

quantite_depots_clients	C1	C2	C3	C4	C5	C6
Newcastle	0	10000	25000	35000	0	0
Birmingham	0	0	0	0	50000	0
Londres	0	0	0	0	10000	20000
Exeter	0	0	0	0	0	0

TABLE 5: Quantité des dépôts aux clients

quantite_usines_clients	C1	C2	C3	C4	C5	C6
Liverpool	50000	0	15000	0	0	0
Brighton	0	0	0	0	0	0

TABLE 6: Quantité des usines aux clients

Ouverture et agrandissement de dépôt

Fichier (bis) : `exercice_2_bis.mod`, `exercice_2_bis.dat`, `exercice_2_bis.run`, `exercice_2_bis.log`

L'entreprise a la possibilité d'ouvrir des dépôts à Bristol et à Northampton. De plus, elle peut agrandir le dépôt de Birmingham et fermer les dépôts de Newcastle et Exeter sachant qu'il n'est pas souhaitable de maintenir plus de quatre dépôts. Ci-dessous, nous pouvons trouver les remboursements d'emprunts destinés à financer l'expansion de l'entreprise.

	coûts	débts
Bristol	12000	30000
Northampton	4000	25000
Birmingham	3000	20000

Nous considérerons le tableau des coûts de distribution suivant.

fournit à	usine Liverpool	usine Brighton	dépôt Bristol	dépôt Northampton
Bristol	0.6	0.4		
Northampton	0.4	0.3		
C1			1.2	
C2			0.6	0.4
C3			0.5	
C4				0.5
C5			0.3	0.6
C6			0.8	0.9

TABLE 7: Coûts de distribution

Objectifs

Nous examinerons si la meilleure stratégie pour minimiser les coûts consiste à construire, agrandir ou fermer des dépôts.

Paramètres

On modifie la paramètre `capa_depart` :

```
param capa_depart := Liverpool 150000 Brighton 200000 Newcastle 70000
    Birmingham 50000 Londres 100000 Exeter 40000 Bristol 30000 Northampton
    25000;
```

Ajout de nouvelles routes :

```
set RouteUC := (Liverpool,C1) (Liverpool,C3) (Liverpool,C4) (Liverpool,C6)
    (Brighton,C1);
set RouteDC := (Newcastle,C2) (Newcastle,C3) (Newcastle,C4) (Newcastle,C6)
    (Birmingham,C1) (Birmingham,C2) (Birmingham,C3) (Birmingham,C4) (Birmingham,C5)
    (Londres,C2) (Londres,C3) (Londres,C5) (Londres,C6)
    (Exeter,C3) (Exeter,C4) (Exeter,C5) (Exeter,C6)
    (Bristol,C1) (Bristol,C2) (Bristol,C3) (Bristol,C5) (Bristol,C6)
    (Northampton,C2) (Northampton,C4) (Northampton,C5) (Northampton,C6);
set RouteUD := (Liverpool,Newcastle) (Liverpool,Birmingham) (Liverpool,Londres) (Liverpool,Exeter) (Liverpool,Bristol)
    (Liverpool,Northampton)
    (Brighton,Birmingham) (Brighton,Londres) (Brighton,Exeter) (Brighton,Bristol) (Brighton,Northampton);
```

Variables supplémentaires

Nous considérons des variables binaires pour décrire l'ouverture ou la fermeture d'un dépôt. Si la variable est égale à 0, alors le dépôt est fermé, sinon il est ouvert. Mais aussi, l'expansion ou non du dépôt de Newcastle est caractérisée par une variable binaire codée de la même façon.

```

var etat_bri := 0, binary; # Variable d'état du dépôt de Bristol
var etat_north := 0, binary; # Variable d'état du dépôt de
    Northampton
var ext_bir := 0, binary; # Variable d'état du dépôt de Birmingham
var etat_exe := 1, binary; # Variable d'état du dépôt d'Exeter
var etat_new := 1, binary; # Variable d'état de l'expansion du
    dépôt de Newcastle

```

Contraintes supplémentaires

1. Vérifier qu'il n'y a pas plus de quatre dépôts ouverts.

```

subject to nombre_depots:
    etat_bri + etat_north + etat_new + etat_exe = 4;

```

2. Mettre toutes les quantités associées aux dépôts fermés à zéro.

```

subject to zero_etat_bri {u in Usines}:
    if (etat_bri = 0) then quantite_usines_depots[u, 'Bristol'] = 0;

subject to zero_etat_north {u in Usines}:
    if (etat_north = 0) then quantite_usines_depots[u, 'Northampton'] = 0;

subject to zero_etat_new {u in Usines}:
    if (etat_new = 0) then quantite_usines_depots[u, 'Newcastle'] = 0;

subject to zero_etat_exe {u in Usines}:
    if (etat_exe = 0) then quantite_usines_depots[u, 'Exeter'] = 0;

```

3. Il faut s'assurer que l'expansion ou non du dépôt de Birmingham soit prise en compte dans les contraintes de débits.

```

subject to debit_limite{d in (Depots diff {'Birmingham'})}:
    sum {u in Usines} quantite_usines_depots[u, d] <=
        capa_depart[d] ;

subject to debit_limite_bir:
    sum {u in Usines} quantite_usines_depots[u, 'Birmingham'] <=
        capa_depart['Birmingham'] + ext_bir*20000;

```

Minimisation

On minimise le coût associé aux chemins de livraison et aux quantités correspondantes. Nous prenons en compte les variables d'état dans le calcul du coût.

```

minimize prix:
    sum{u1 in Usines,d1 in Depots}
        quantite_usines_depots[u1,d1]*prix_routes[u1,d1]
    + sum{d2 in Depots,c2 in Clients}
        quantite_depots_clients[d2,c2]*prix_routes[d2,c2]
    + sum{u3 in Usines,c3 in Clients}
        quantite_usines_clients[u3,c3]*prix_routes[u3,c3]
    - (1-etat_new) * 10000 - (1-etat_exe) * 5000
    + etat_bri * 12000 + etat_north * 4000 + ext_bir * 3000;

```

Résultats

1. Dans un premier temps, sans prendre en compte les préférences des clients

quantite_usines_depots	Birmingham	Exeter	Londres	Newcastle	Bristol	Northampton
Liverpool	0	40000	0	0	0	0
Brighton	70000	0	10000	0	0	25000

TABLE 8: Quantité des usines aux dépôts

quantite_depots_clients	C1	C2	C3	C4	C5	C6
Newcastle	0	0	0	0	0	0
Birmingham	0	10000	0	10000	50000	0
Londres	0	0	0	0	10000	0
Exeter	0	0	40000	0	0	0
Bristol	0	0	0	0	0	0
Northampton	0	0	0	25000	0	0

TABLE 9: Quantité des dépôts aux clients

quantite_usines_clients	C1	C2	C3	C4	C5	C6
Liverpool	50000	0	0	0	0	20000
Brighton	0	0	0	0	0	0

TABLE 10: Quantité des usines aux clients

Le coût est de **174000**.

Variables d'état en sortie d'optimisation :

- `etat_bri` = 0
- `etat_north` = 1
- `etat_new` = 0
- `etat_exe` = 1
- `ext_bir` = 1

Conclusion

Dans le cas où nous ne tenons pas compte des préférences des clients, afin d'optimiser les coûts de distribution, nous devons agrandir le dépôt de Birmingham, maintenir ouvert le dépôt d'Exeter, ouvrir le dépôt de Northampton et fermer le dépôt de Newcastle. Le coût de distribution associé est de **174 000** livres.

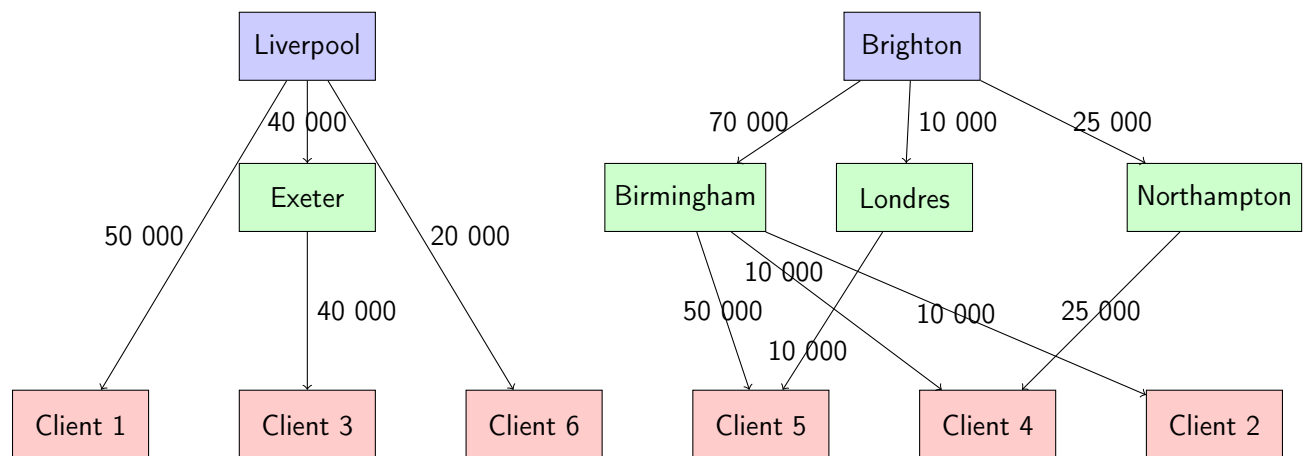


FIGURE 2 – Schéma de distribution des produits (quantités en tonnes)

2. Dans un second temps, en considérant les préférences des clients

Modifications

On ajoute les contraintes suivantes, qui permettent de rendre nulles les quantités associées aux chemins de livraison non souhaités.

Fichier .mod

```

subject to satisfaction_preferences_Usines {u in Usines, c in
  Clients}:
  if(preferences[c,u]=0) then quantite_usines_clients[u,c]=0;

subject to satisfaction_preferences_Depots {d in Depots, c in
  Clients}:
  if(preferences[c,d]=0) then quantite_depots_clients[d,c]=0;

```

On modifie le paramètre **preferences**.

Fichier .dat

```

param preferences : Liverpool Brighton Newcastle Birmingham Londres Exeter Bristol Northampton:=
  C1      1      0      0      0      0      0      0      0
  C2      0      0      1      0      0      0      0      0
  C3      1      1      1      1      1      1      1      1
  C4      1      1      1      1      1      1      1      1
  C5      0      0      0      1      0      0      0      0
  C6      0      0      0      0      1      1      0      0;

```

Résultats

quantite_usines_depots	Birmingham	Exeter	Londres	Newcastle	Bristol	Northampton
Liverpool	0	0	0	50000	0	0
Brighton	70000	0	20000	0	0	25000

TABLE 11: Quantité des usines aux dépôts

quantite_depots_clients	C1	C2	C3	C4	C5	C6
Newcastle	0	10000	40000	0	0	0
Birmingham	0	0	0	10000	60000	0
Londres	0	0	0	0	0	20000
Exeter	0	0	0	0	0	0
Bristol	0	0	0	0	0	0
Northampton	0	0	0	25000	0	0

TABLE 12: Quantité des dépôts aux clients

quantite_usines_clients	C1	C2	C3	C4	C5	C6
Liverpool	50000	0	0	0	0	0
Brighton	0	0	0	0	0	0

TABLE 13: Quantité des usines aux clients

Le coût est de **233000**.

Variables d'état en sortie d'optimisation :

- **etat_bri** = 0
- **etat_north** = 1
- **etat_new** = 1
- **etat_exe** = 0
- **ext_bir** = 1

Conclusion

Nous constatons qu'il est suffisant d'agrandir le dépôt de Birmingham, de fermer celui d'Exeter, et d'ouvrir le dépôt de Northampton pour minimiser les coûts de livraison tout en respectant les préférences des clients. Le coût de distribution associé est de **233 000** livres.

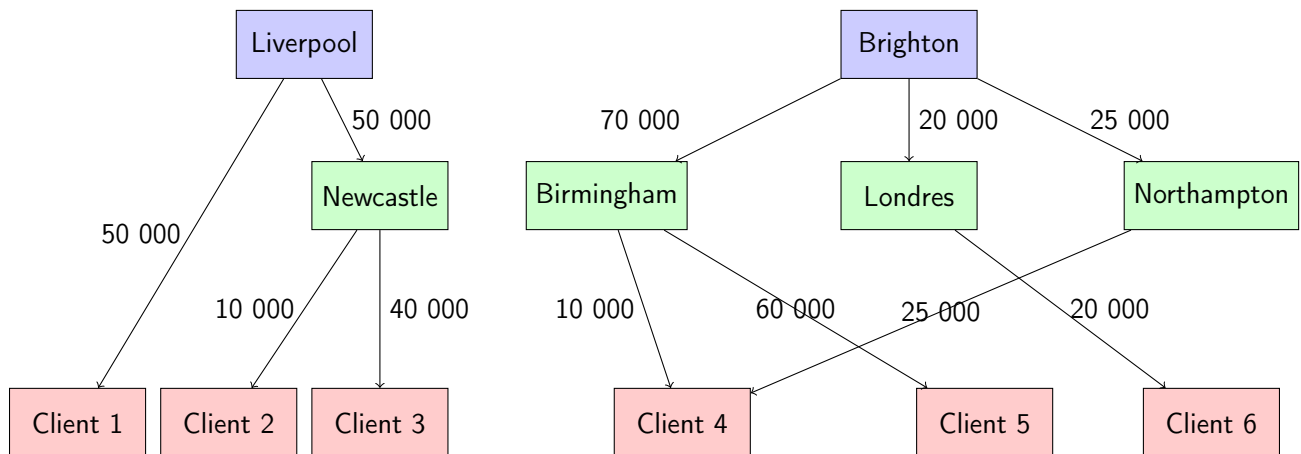


FIGURE 3 – Schéma de distribution des produits (quantités en tonnes)

Annexes (Code AMPL avec les résultats)

Fichier .mod

```
set Clients; # Liste des clients
set Depots; # Liste des dépôts
set Usines; # Liste des usines
set Usines_Depots := Usines union Depots;
set Depots_Clients := Depots union Clients;

# Ici, on crée deux ensembles, Usines_Depots et Depots_Clients, pour
# simplifier l'écriture des contraintes par la suite.

set CAT := Usines union Depots union Clients;

# CAT sert simplement à contenir les usines, dépôts et clients pour
# construire les ensembles ci-dessous.

set RouteUC within CAT cross CAT; set ZUC := (Usines cross Clients) diff
RouteUC;
# RouteUC correspond à tous les chemins de livraison possibles entre les
# usines et les clients.
set RouteDC within CAT cross CAT; set ZDC := (Depots cross Clients) diff
RouteDC;
# RouteDC correspond à tous les chemins de livraison possibles entre les
# dépôts et les clients.
set RouteUD within CAT cross CAT; set ZUD := (Usines cross Depots) diff
RouteUD;
# RouteUD correspond à tous les chemins de livraison possibles entre les
# usines et les dépôts.

# ZUC, ZDC et ZUD correspondent aux chemins de livraison impossibles
# respectivement contraires aux ensembles RouteUC, RouteDC et RouteUD.

param capa_depart{Usines_Depots};
# Contient les capacités de production des usines et les débits mensuels
# limités des dépôts.
param besoin_clients{Clients};
# Contient les besoins des clients.
param prix_routes{Usines_Depots,Depots_Clients};
# Stocke tous les prix.
param preferences{Clients,Usines_Depots};

var quantite_usines_depots{Usines,Depots} >= 0;
var quantite_depots_clients{Depots,Clients} >= 0;
var quantite_usines_clients{Usines,Clients} >= 0;

## Enlevez le code ci-dessous des commentaires si vous souhaitez maximiser
## selon les préférences.

#maximize satisfaction_preferences;
```



```

#      sum{u in Usines,c in Clients}
quantite_usines_clients[u,c]*preferences[c,u]
#      +sum{d in Depots,c in Clients}
quantite_depots_clients[d,c]*preferences[c,d];

minimize cout:
    sum{u1 in Usines,d1 in Depots}
        quantite_usines_depots[u1,d1]*prix_routes[u1,d1]
+    sum{d2 in Depots,c2 in Clients}
        quantite_depots_clients[d2,c2]*prix_routes[d2,c2]
+    sum{u3 in Usines,c3 in Clients}
        quantite_usines_clients[u3,c3]*prix_routes[u3,c3];

subject to capacite_limite{u in Usines}:
    sum {d in Depots} quantite_usines_depots[u,d]
+sum {c in Clients} quantite_usines_clients[u,c] <= capa_depart[u];

subject to debit_limite{d in Depots}:
    sum {u in Usines} quantite_usines_depots[u,d] <= capa_depart[d];

subject to satisfaction_besoin{c in Clients}:
    sum {d in Depots} quantite_depots_clients[d,c]
+    sum {u in Usines} quantite_usines_clients[u,c] == besoin_clients[c];

# La quantité de produit envoyée des usines au dépôt doit tre égale à la
# quantité de produit envoyée du dépôt aux clients

subject to bilan{d in Depots}:
    sum {u in Usines} quantite_usines_depots[u,d]
    = sum {c in Clients} quantite_depots_clients[d,c] ;

# Les quantités associées aux chemins de livraison qui ne sont pas possibles
# doivent tre nulles.

subject to zero_DC {(p,c) in ZDC} :
    quantite_depots_clients[p,c] =0;

subject to zero_UC {(d,k) in ZUC}:
    quantite_usines_clients[d,k] = 0;

subject to zero_UD {(k,l) in ZUD}:
    quantite_usines_depots[k,l] = 0;
#

```

Fichier .dat

```

set Clients := C1 C2 C3 C4 C5 C6;
set Depots := Newcastle Birmingham Londres Exeter;
set Usines := Liverpool Brighton;

set RouteUC := (Liverpool,C1) (Liverpool,C3) (Liverpool,C4) (Liverpool,C6)
               (Brighton,C1);
set RouteDC := (Newcastle,C2) (Newcastle,C3) (Newcastle,C4) (Newcastle,C6)
               (Birmingham,C1) (Birmingham,C2) (Birmingham,C3)
               (Birmingham,C4) (Birmingham,C5)
               (Londres,C2) (Londres,C3) (Londres,C5) (Londres,C6)
               (Exeter,C3) (Exeter,C4) (Exeter,C5) (Exeter,C6);
set RouteUD := (Liverpool,Newcastle) (Liverpool,Birmingham)
               (Liverpool,Londres) (Liverpool,Exeter)
               (Brighton,Birmingham) (Brighton,Londres) (Brighton,Exeter);

param prix_routes : Newcastle Birmingham Londres Exeter C1 C2 C3 C4 C5 C6 :=
    Liverpool      0.5      0.5      1      0.2  1      0  1.5
      2      0      1.0
    Brighton      0      0.3      0.5  0.2  2      0      0
      0      0      0
    Newcastle      0      0      0      0      0      1.5
      0.5 1.5 0      1
    Birmingham      0      0      0      0      1      0.5
      0.5 1  0.5  0
    Londres      0      0      0      0      0      1.5 2      0
      0.5 1.5
    Exeter      0      0      0      0      0      0      0.2 1.5
      0.5 1.5;

param capa_depart := Liverpool 150000 Brighton 200000 Newcastle 70000
    Birmingham 50000 Londres 100000 Exeter 40000;
param besoin_clients := C1 50000 C2 10000 C3 40000 C4 35000 C5 60000 C6 20000;

param preferences : Liverpool Brighton Newcastle Birmingham Londres Exeter :=
    C1      1      0      0      0      0      0
    C2      0      0      1      0      0      0
    C3      1      1      1      1      1      1
    C4      1      1      1      1      1      1
    C5      0      0      0      1      0      0
    C6      0      0      0      0      1      1;
#

```

Fichier .run

```
# automatically generated on Thu Oct 03 2024 at 12:21:11 PM CEST
reset;
model ../../workspace/ampl/dm/exercice_2.mod;
data ../../workspace/ampl/dm/exercice_2.dat;
option solver cplex;
solve;

display quantite_usines_depots;
display quantite_depots_clients;
display quantite_usines_clients;
```

Fichier .log

```
include ../../workspace/ampl/dm/exercice_2.run

Presolve eliminates 16 constraints and 15 variables.
Adjusted problem:
29 variables, all linear
15 constraints, all linear; 64 nonzeros
1 linear objective; 29 nonzeros.

solve
optimal solution; objective 198500
14 dual simplex iterations (0 in phase I)
display
quantite_usines_depots :=
Brighton Birmingham 50000
Brighton Exeter      0
Brighton Londres    55000
Brighton Newcastle   0
Liverpool Birmingham 0
Liverpool Exeter     40000
Liverpool Londres    0
Liverpool Newcastle  0
;

display
quantite_depots_clients [*,*] (tr)
: Birmingham Exeter Londres Newcastle :=
C1      0      0      0      0
C2  10000      0      0      0
C3      0  40000      0      0
C4  35000      0      0      0
C5   5000      0  55000      0
C6      0      0      0      0
;

display
quantite_usines_clients :=
Brighton C1      0
Brighton C2      0
Brighton C3      0
Brighton C4      0
Brighton C5      0
Brighton C6      0
Liverpool C1 50000
Liverpool C2      0
Liverpool C3      0
Liverpool C4      0
Liverpool C5      0
Liverpool C6 20000
;
```

```
# ampl: include ../../workspace/ampl/dm/exercice_2.run
```

Presolve eliminates 16 constraints and 15 variables.

Adjusted problem:

29 variables, **all** linear

15 constraints, **all** linear; 64 nonzeros

2 objectives, **all** linear; 43 nonzeros.

solve

optimal solution; objective 205000

8 dual simplex iterations (0 **in** phase I)

solve

Objective = satisfaction_preferences

display

quantite_usines_depots :=

Brighton Birmingham 50000

Brighton Exeter 0

Brighton Londres 15000

Brighton Newcastle 0

Liverpool Birmingham 0

Liverpool Exeter 0

Liverpool Londres 15000

Liverpool Newcastle 70000

;

display

quantite_depots_clients [*,*] (tr)

: Birmingham Exeter Londres Newcastle :=

C1	0	0	0	0
----	---	---	---	---

C2	0	0	0	10000
----	---	---	---	-------

C3	0	0	0	25000
----	---	---	---	-------

C4	0	0	0	35000
----	---	---	---	-------

C5	50000	0	10000	0
----	-------	---	-------	---

C6	0	0	20000	0
----	---	---	-------	---

;

display

quantite_usines_clients :=

Brighton C1 0

Brighton C2 0

Brighton C3 0

Brighton C4 0

Brighton C5 0

Brighton C6 0

Liverpool C1 50000

Liverpool C2 0

Liverpool C3 15000

Liverpool C4 0

Liverpool C5 0

Liverpool C6 0

;

Fichier .mod (bis)

```
# ----- Le solveur utilisé pour les résultats du compte
rendu est gurobi ----- #

set Clients; # Liste des clients
set Depots; # Liste des dépôts
set Usines; # Liste des usines
set Usines_Depots := Usines union Depots;
set Depots_Clients := Depots union Clients;

# Ici, on crée deux ensembles, Usines_Depots et Depots_Clients, pour
simplifier l'écriture des contraintes par la suite.

set CAT := Usines union Depots union Clients;

# CAT sert simplement à contenir les usines, dépôts et clients pour
construire les ensembles ci-dessous.

set RouteUC within CAT cross CAT; set ZUC := (Usines cross Clients) diff
RouteUC;
# RouteUC correspond à tous les chemins de livraison possibles entre les
usines et les clients.
set RouteDC within CAT cross CAT; set ZDC := (Depots cross Clients) diff
RouteDC;
# RouteDC correspond à tous les chemins de livraison possibles entre les
dépôts et les clients.
set RouteUD within CAT cross CAT; set ZUD := (Usines cross Depots) diff
RouteUD;
# RouteUD correspond à tous les chemins de livraison possibles entre les
usines et les dépôts.

# ZUC, ZDC et ZUD correspondent aux chemins de livraison impossibles
respectivement contraires aux ensembles RouteUC, RouteDC et RouteUD.

param capa_depart{Usines_Depots};
# Contient les capacités de production des usines et les débits mensuels
limités des dépôts.
param besoin_clients{Clients};
# Contient les besoins des clients.
param prix_routes{Usines_Depots,Depots_Clients};
# Stocke tous les prix.
param preferences{Clients,Usines_Depots};

var quantite_usines_depots{Usines,Depots} >= 0;
var quantite_depots_clients{Depots,Clients} >= 0;
var quantite_usines_clients{Usines,Clients} >= 0;

var etat_bri := 0, binary; # Variable état du dépôt de Bristol
```

```

var etat_north := 0, binary; # Variable état du dépôt de Northampton
var ext_bir := 0, binary; # Variable état du dépôt de Birmingham
var etat_exe := 1, binary; # Variable état du dépôt dExeter
var etat_new := 1, binary; # Variable état de l'expansion du dépôt de
    Newcastle

minimize prix:
    sum{u1 in Usines,d1 in Depots}
        quantite_usines_depots[u1,d1]*prix_routes[u1,d1]
    + sum{d2 in Depots,c2 in Clients}
        quantite_depots_clients[d2,c2]*prix_routes[d2,c2]
    + sum{u3 in Usines,c3 in Clients}
        quantite_usines_clients[u3,c3]*prix_routes[u3,c3]
    - (1-etat_new) * 10000 - (1-etat_exe) * 5000
    + etat_bri * 12000 + etat_north * 4000 + ext_bir * 3000;

subject to capacite_limite{u in Usines}:
    sum {d in Depots} quantite_usines_depots[u,d]
    + sum {c in Clients} quantite_usines_clients[u,c] <= capa_depart[u];

subject to debit_limite{d in (Depots diff {'Birmingham'})}:
    sum {u in Usines} quantite_usines_depots[u,d] <= capa_depart[d] ;

subject to debit_limite_bir:
    sum {u in Usines} quantite_usines_depots[u,'Birmingham'] <=
        capa_depart['Birmingham']+ ext_bir*20000;

subject to satisfaction_besoin{c in Clients}:
    sum {d in Depots} quantite_depots_clients[d,c]
    + sum {u in Usines} quantite_usines_clients[u,c] == besoin_clients[c];

# La quantité de produit envoyée des usines au dépôt doit tre égale à la
    quantité de produit envoyée du dépôt aux clients

subject to bilan{d in Depots}:
    sum {u in Usines} quantite_usines_depots[u,d]
    = sum {c in Clients} quantite_depots_clients[d,c] ;

# Pour vérifier qu'il ny a pas plus de quatre dépôts ouverts.

subject to nombre_depots:
    etat_bri + etat_north + etat_new+ etat_exe=2;

# Les quantités associées aux chemins de livraison qui ne sont pas possibles
    doivent tre nulles.

subject to zero_DC {(p,c) in ZDC} :
    quantite_depots_clients[p,c] =0;

```

```

subject to zero_UC {(d,k) in ZUC}:
    quantite_usines_clients[d,k] = 0;

subject to zero_UD {(k,l) in ZUD}:
    quantite_usines_depots[k,l] = 0;

# Toutes les quantités associées aux dépôts fermés à zéro.

subject to zero_etat_bri {c in Clients}:
    if (etat_bri = 0) then quantite_depots_clients['Bristol',c] =0;

subject to zero_etat_north {c in Clients}:
    if (etat_north = 0) then quantite_depots_clients['Northampton',c] =0;

subject to zero_etat_new {c in Clients}:
    if (etat_new = 0) then quantite_depots_clients['Newcastle',c] =0;

subject to zero_etat_exe {c in Clients}:
    if (etat_exe = 0) then quantite_depots_clients['Exeter',c] =0;

## Enlevez le code ci-dessous des commentaires si vous souhaitez prendre en
compte les préférences.

#subject to satisfaction_preferences_usines{u in Usines, c in Clients}:
#    if (preferences[c,u]=0) then quantite_usines_clients[u,c]=0;

#subject to satisfaction_preferences_depots{d in Depots, c in Clients}:
#    if (preferences[c,d]=0) then quantite_depots_clients[d,c]=0;
#

```

Fichier .dat (bis)

```

set Clients := C1 C2 C3 C4 C5 C6;
set Depots := Newcastle Birmingham Londres Exeter Bristol Northampton;
set Usines := Liverpool Brighton;

set RouteUC := (Liverpool,C1) (Liverpool,C3) (Liverpool,C4) (Liverpool,C6)
               (Brighton,C1);
set RouteDC := (Newcastle,C2) (Newcastle,C3) (Newcastle,C4) (Newcastle,C6)
               (Birmingham,C1) (Birmingham,C2) (Birmingham,C3)
               (Birmingham,C4) (Birmingham,C5)
               (Londres,C2) (Londres,C3) (Londres,C5) (Londres,C6)
               (Exeter,C3) (Exeter,C4) (Exeter,C5) (Exeter,C6)
               (Bristol,C1) (Bristol,C2) (Bristol,C3) (Bristol,C5)
               (Bristol, C6)
               (Northampton,C2) (Northampton,C4) (Northampton,C5)
               (Northampton,C6);
set RouteUD := (Liverpool,Newcastle) (Liverpool,Birmingham)
               (Liverpool,Londres) (Liverpool,Exeter) (Liverpool,Bristol)
               (Liverpool,Northampton)
               (Brighton,Birmingham) (Brighton,Londres) (Brighton,Exeter)
               (Brighton, Bristol) (Brighton,Northampton);

param prix_routes : Newcastle Birmingham Londres Exeter C1 C2 C3 C4 C5 C6
Bristol Northampton:=
    Liverpool      0.5      0.5      1      0.2  1      0  1.5
      2      0      1.0      0.6  0.4
    Brighton      0      0.3      0.5  0.2  2      0      0
      0      0      0      0.4  0.3
    Newcastle      0      0      0      0      1.5
      0.5 1.5 0      1      0      0
    Birmingham      0      0      0      0      1      0.5
      0.5 1  0.5 0      0      0
    Londres      0      0      0      0      1.5 2      0
      0.5 1.5 0      0
    Exeter      0      0      0      0      0      0.2 1.5
      0.5 1.5 0      0
    Bristol      0      0      0      1.2      0.6      0.5
      0      0.3  0.8 0      0
    Northampton      0      0      0      0      0      0.4
      0      0.5  0.6 0.9 0      0;

param capa_depart := Liverpool 150000 Brighton 200000 Newcastle 70000
    Birmingham 50000 Londres 100000 Exeter 40000 Bristol 30000 Northampton
    25000;
param besoin_clients := C1 50000 C2 10000 C3 40000 C4 35000 C5 60000 C6 20000;

param preferences : Liverpool Brighton Newcastle Birmingham Londres Exeter
    Bristol Northampton:=

```

C1		1	0	0	0	0	0
0	0						
C2		0	0	1	0	0	0
0	0						
C3		1	1	1	1	1	1
1	1						
C4		1	1	1	1	1	1
1	1						
C5		0	0	0	1	0	0
0	0						
C6		0	0	0	0	1	1
0	0;						

#



Fichier .run (bis)

```
# automatically generated on Thu Oct 03 2024 at 12:21:11 PM CEST
reset;
model ../../workspace/ampl/dm/exercice_2_bis.mod;
data ../../workspace/ampl/dm/exercice_2_bis.dat;
option solver gurobi;
solve;

display quantite_usines_depots;
display quantite_depots_clients;
display quantite_usines_clients;
display etat_bri;
display etat_north;
display etat_new;
display etat_exe;
display ext_bir;
```

Fichier .log (bis)

```
include ../../workspace/ampl/dm/exercice_2_bis.run

Presolve eliminates 19 constraints and 18 variables.
Adjusted problem:
47 variables:
    1 binary variable
    21 nonlinear variables
    25 linear variables
44 constraints; 140 linear nonzeros
    24 nonlinear constraints
    20 linear constraints
1 linear objective; 47 nonzeros.

solve
Gurobi 11.0.3: optimal solution; objective 174000
21 simplex iterations
1 branching node
display
quantite_usines_depots :=
Brighton Birmingham 70000
Brighton Bristol      0
Brighton Exeter       0
Brighton Londres     10000
Brighton Newcastle    0
Brighton Northampton 25000
Liverpool Birmingham  0
Liverpool Bristol     0
Liverpool Exeter     40000
Liverpool Londres     0
Liverpool Newcastle   0
Liverpool Northampton 0
;

display
quantite_depots_clients [*,*]
:      C1  C2  C3  C4  C5  C6  :=
Birmingham 0 10000 0 10000 50000 0
Bristol     0 0 0 0 0 0
Exeter      0 0 40000 0 0 0
Londres     0 0 0 0 10000 0
Newcastle   0 0 0 0 0 0
Northampton 0 0 0 25000 0 0
;

display
quantite_usines_clients :=
Brighton C1  0
Brighton C2  0
Brighton C3  0
```

```

Brighton C4    0
Brighton C5    0
Brighton C6    0
Liverpool C1 50000
Liverpool C2    0
Liverpool C3    0
Liverpool C4    0
Liverpool C5    0
Liverpool C6 20000
;

display
etat_bri = 0

display
etat_north = 1

display
etat_new = 0

display
etat_exe = 1

display
ext_bir = 1
# ampl: include ../../workspace/ampl/dm/exercice_2_bis.run

Presolve eliminates 70 constraints and 36 variables.
Adjusted problem:
29 variables:
    1 binary variable
    11 nonlinear variables
    17 linear variables
41 constraints; 94 linear nonzeros
    24 nonlinear constraints
    17 linear constraints
1 linear objective; 29 nonzeros.

solve
Gurobi 11.0.3: optimal solution; objective 233000
12 simplex iterations
1 branching node
display
quantite_usines_depots :=
Brighton Birmingham 70000
Brighton Bristol    0
Brighton Exeter     0
Brighton Londres    20000
Brighton Newcastle  0
Brighton Northampton 25000
Liverpool Birmingham 0

```

```

Liverpool Bristol      0
Liverpool Exeter       0
Liverpool Londres      0
Liverpool Newcastle 50000
Liverpool Northampton 0
;

display
quantite_depots_clients [*,*]
:      C1   C2   C3   C4   C5   C6   :=
Birmingham 0     0     0 10000 60000 0
Bristol     0     0     0    0     0    0
Exeter      0     0     0    0     0    0
Londres     0     0     0    0     0 20000
Newcastle  0 10000 40000    0     0    0
Northampton 0     0     0 25000    0    0
;

display
quantite_usines_clients :=
Brighton C1    0
Brighton C2    0
Brighton C3    0
Brighton C4    0
Brighton C5    0
Brighton C6    0
Liverpool C1 50000
Liverpool C2  0
Liverpool C3  0
Liverpool C4  0
Liverpool C5  0
Liverpool C6  0
;

display
etat_bri = 0

display
etat_north = 1

display
etat_new = 1

display
etat_exe = 0

display
ext_bir = 1
# ampl:

```
